포팅메뉴얼

온길(Ongil) 프로젝트 배포 가이드

목차

1. 개발 환경

Frontend

- JavaScript
- React 18
- Vite
- Axios
- React Router
- [기타 프론트엔드 라이브러리]

Backend

- Java 17
- Spring Boot 3.5.6
- Spring WebFlux (또는 Spring MVC)
- Spring Security
- Spring Data JPA
- Spring Data Redis
- JWT
- Gradle 8.x
- WebRTC
- STOMP/WebSocket
- RabbitMQ

Database

- PostgreSQL (AWS RDS)
- Redis (AWS ElastiCache)
- AWS S3

서버 배포 환경

- AWS EC2 (3대 - Docker Swarm 클러스터)
- Docker Swarm
- Traefik (EC2마다 1개씩, 리버스 프록시 & 로드 밸런서)
- SSL (Let's Encrypt 또는 인증서)

CI/CD

- Jenkins
- Docker Hub private registry
- Github private registry
- GitLab

모니터링

- Prometheus
- Grafana
- Loki
- cAdvisor
- Node Exporter

협업 및 형상 관리

- GitLab
- Jira
- Notion

📌 2. 환경 변수 설정

Frontend

`.env`

```
BASE_URL = https://staging.on-gil.co.kr/
TMAP_API_KEY=my-tmap-key
SSE_URL = https://staging.on-gil.co.kr/api/v1/location/stream
```

Backend

`application.yml` **(주요 설정)**

```
spring:
  application:
    name: ongil
  profiles:
    active: ${SPRING_PROFILES_ACTIVE}
```

```
  # Environment Variables
  config:
    import: optional:file:.env.${spring.profiles.active}[.properties]

  # PostgreSQL (AWS RDS)
  datasource:
    driver-class-name: org.postgresql.Driver
    url: jdbc:postgresql://${DB_HOST}:${DB_PORT}/${DB_NAME}?
sslmode=${DB_SSL_MODE:require}&reWriteBatchedInserts=true
    username: ${DB_USERNAME}
    password: ${DB_PASSWORD}
    hikari:
      maximum-pool-size: ${DB_POOL_MAX_SIZE:10}
      minimum-idle: ${DB_POOL_MIN_IDLE:2}
      connection-timeout: 30000
      idle-timeout: 600000
      max-lifetime: 1800000

  # RabbitMQ
  rabbitmq:
    host: ${RABBITMQ_HOST:localhost}
    port: ${RABBITMQ_PORT:5672}
    username: ${RABBITMQ_USERNAME:guest}
    password: ${RABBITMQ_PASSWORD:guest}
    # 선택적 설정
    virtual-host: /
    connection-timeout: 10000
    publisher-confirm-type: correlated
    publisher-returns: true

  # JPA/Hibernate
  jpa:
    open-in-view: false
    hibernate:
      ddl-auto: update
    properties:
      hibernate:
        format_sql: true
        show_sql: false
        jdbc:
          time_zone: Asia/Seoul

  # Redis (AWS ElastiCache)
  data:
    redis:
      host: ${REDIS_HOST}
      port: ${REDIS_PORT:6379}
      # password: ${REDIS_PASSWORD:}
      ssl:
        enabled: ${REDIS_SSL_ENABLED}

  cache:
    type: redis
```

```yaml
    redis:
      time-to-live: 600000 # 10? TTL ??

  # File Upload
  servlet:
    multipart:
      max-file-size: 25MB
      max-request-size: 25MB

  # S3 (Spring Cloud AWS)
  cloud:
    aws:
      region:
        static: ${AWS_REGION}
      credentials:
        access-key: ${AWS_ACCESS_KEY}
        secret-key: ${AWS_SECRET_KEY}
      s3:
        bucket: ${S3_BUCKET}

management:
  endpoints:
    web:
      exposure:
        include: health,info,env,configprops,prometheus
  endpoint:
    env:
      enabled: true
    configprops:
      enabled: true
    prometheus:
      enabled: true
  prometheus:
    metrics:
      export:
        enabled: true



decorator:
  datasource:
    p6spy:
      enable-logging: false        # 로깅 활성화
      multiline: true             # SQL을 여러 줄로 예쁘게
      logging: slf4j              # SLF4J로 로깅
      tracing:
        include-parameter-values: true  # 바인딩 파라미터 값 표시

# Custom Properties
ongil:
  aws:
    region: ${AWS_REGION}
    s3:
      bucket: ${S3_BUCKET}
```

```yaml
# Server Configuration
server:
  port: ${SERVER_PORT:8080}
  forward-headers-strategy: framework  # HTTPS/프록시 환경에서 X-Forwarded-* 헤더 신뢰
  servlet:
    encoding:
      charset: UTF-8
      enabled: true
      force: true

logging:
  level:
    root: INFO
    org.hibernate.SQL: WARN
    p6spy: info
    org.redisson: WARN

jwt:
  secret: ${JWT_SECRET}
  access-token-expiration: ${JWT_ACCESS_TOKEN_EXPIRATION:86400000}
  refresh-token-expiration: ${JWT_REFRESH_TOKEN_EXPIRATION:259200000}
  verification: # phone verification token
    expiration: 300000 # 5 minutes

tmap:
  app-key: ${TMAP_APP_KEY}
  api:
    base-url: https://apis.openapi.sk.com

coolsms:
  api:
    key: ${COOLSMS_API_KEY}        # CoolSMS API KEY
    secret: ${COOLSMS_API_SECRET}  # coolSMS API SECRET
  from: ${COOLSMS_FROM_NUMBER}     # Sender Phone Number
  domain: https://api.coolsms.co.kr


springdoc:
  api-docs:
    path: /v3/api-docs
  swagger-ui:
    path: /swagger-ui.html
    enabled: true

# TURN/STUN Server Configuration
turn:
  server:
    host: ${TURN_SERVER_HOST:43.202.55.40}
    port: ${TURN_SERVER_PORT:3478}
  shared-secret: ${TURN_SHARED_SECRET:your-turn-secret-key-here}
  ttl: ${TURN_TTL:3600}  # 1시간
```

```
# CORS Configuration
cors:
  allowed-origins: ${CORS_ALLOWED_ORIGINS:*}  # local/dev: *, prod: 실제 도메인
  allowed-methods: ${CORS_ALLOWED_METHODS:GET,POST,PUT,DELETE,PATCH,OPTIONS}
  allowed-headers: ${CORS_ALLOWED_HEADERS:*}
  allow-credentials: ${CORS_ALLOW_CREDENTIALS:true}
  max-age: ${CORS_MAX_AGE:3600}

# Call Configuration
call:
  timeout-seconds: ${CALL_TIMEOUT_SECONDS:60}  # 통화 타임아웃 (초), 기본 60초

# GMS Configuration (SSAFY Generative Model Service)
# 다양한 AI 모델 통합 지원
gms:
  api:
    key: ${GMS_API_KEY}                # 통합 API 키 (환경변수)
    timeout: 90                        # 타임아웃 (초)

    # 현재 사용할 Provider와 Model 선택
    provider: openai                   # openai | gemini | claude | runway
    model: gpt-4.1-mini               # 선택한 provider의 모델명

    # Provider별 설정
    providers:
      # OpenAI (텍스트 생성) - 가장 가성비 좋음
      openai:
        base-url: https://gms.ssafy.io/gmsapi/api.openai.com/v1
        endpoint: /chat/completions
        auth-header: Authorization
        auth-prefix: Bearer
        models:
          - gpt-4.1-mini          # Input: 0.004, Output: 0.016 (추천)
          - gpt-4.1-nano          # 더 저렴
          - gpt-4o-mini           # Input: 0.0015, Output: 0.006 (더 저렴)
          - gpt-4o                # 고성능
          - o3-mini               # 추론 특화

      # Google Gemini (텍스트 생성) - 가장 빠르고 저렴
      gemini:
        base-url:
https://gms.ssafy.io/gmsapi/generativelanguage.googleapis.com/v1beta
        endpoint: /models
        auth-type: query          # 쿼리스트링에 key 추가
        models:
          - gemini-2.5-flash-lite # Input: 0.001, Output: 0.004 (가장 저렴)
          - gemini-2.5-flash      # 균형형
          - gemini-2.5-pro        # 고성능

      # Anthropic Claude (텍스트 생성) - 가장 강력
      claude:
        base-url: https://gms.ssafy.io/gmsapi/api.anthropic.com/v1
        endpoint: /messages
        auth-header: x-api-key
```

```
      version-header: anthropic-version
      version: 2023-06-01
      models:
        - claude-sonnet-4-20250514  # Input: 0.03, Output: 0.15 (강력하지만 비쌈)
        - claude-3-7-sonnet-latest
        - claude-3-5-haiku-latest

    # Runway (비디오 생성) - 매우 비쌈, 신중한 사용 필요
    runway:
      base-url: https://gms.ssafy.io/gmsapi/api.dev.runwayml.com/v1
      endpoint: /image_to_video
      auth-header: Authorization
      auth-prefix: Bearer
      version-header: X-Runway-Version
      version: 2024-11-06
      models:
        - gen3a_turbo           # Input: 1, Output: 4000 (매우 비쌈!)
```

환경 변수 목록 (`.env` 파일)

```
# Database
DB_HOST=a305-db.cz22acwewmnw.ap-northeast-2.rds.amazonaws.com
DB_PORT=5432
DB_NAME=ongil
DB_USERNAME=postgres
DB_PASSWORD=your_password
DB_SSL_MODE=require
DB_POOL_MAX_SIZE=10
DB_POOL_MIN_IDLE=2

# Redis
REDIS_HOST=a305-cache-ui6bqx.serverless.apn2.cache.amazonaws.com
REDIS_PORT=6379
REDIS_PASSWORD=
REDIS_SSL_ENABLED=true

# AWS
AWS_REGION=ap-northeast-2
AWS_ACCESS_KEY=your_access_key
AWS_SECRET_KEY=your_secret_key
S3_BUCKET=a305-bucket

# JWT
JWT_SECRET=your_jwt_secret_key
JWT_ACCESS_TOKEN_EXPIRATION=86400000
JWT_REFRESH_TOKEN_EXPIRATION=259200000

# TMAP
TMAP_APP_KEY=your_tmap_key

# COOLSMS
COOLSMS_API_KEY=your_coolsms_key
```

```
COOLSMS_API_SECRET=your_coolsms_secret
COOLSMS_FROM_NUMBER=01012345678

# TURN Server
TURN_SERVER_HOST=turn.on-gil.co.kr
TURN_SERVER_PORT=3478
TURN_SHARED_SECRET=your_turn_secret
TURN_TTL=3600

# RabbitMQ
RABBITMQ_HOST=rabbitmq
RABBITMQ_PORT=5672
RABBITMQ_STOMP_PORT=61613
RABBITMQ_USERNAME=admin
RABBITMQ_PASSWORD=password

# CORS
CORS_ALLOWED_ORIGINS=https://on-gil.co.kr,https://www.on-
gil.co.kr,https://staging.on-gil.co.kr
CORS_ALLOWED_METHODS=GET,POST,PUT,DELETE,PATCH,OPTIONS
CORS_ALLOWED_HEADERS=Authorization,Content-Type,X-Requested-With
CORS_ALLOW_CREDENTIALS=true
CORS_MAX_AGE=3600

# Google Maps
GMS_API_KEY=your_gms_key

# Spring
SPRING_PROFILES_ACTIVE=dev
SERVER_PORT=8080
```

📌 3. 배포 환경 설정

서버 구성

본 프로젝트는 **총 4대의 EC2 인스턴스**를 사용합니다:

1. **인프라 서버 (1대)**: Jenkins, Prometheus, Grafana, Loki 등 모니터링 및 CI/CD 인프라
2. **서비스 서버 (3대)**: Docker Swarm 클러스터로 구성된 실제 애플리케이션 서버+coturn 서버

인프라 서버 접속

```
ssh -i k13a305.pem ubuntu@k13a305.p.ssafy.io
```

서비스 서버 (Swarm Cluster) 접속

Manager Node (Node 1)

```

```

```
ssh -i your-key.pem ubuntu@13.125.112.26
```

Worker Node 2

```
ssh -i your-key.pem ubuntu@43.201.195.106
```

Worker Node 3

```
ssh -i your-key.pem ubuntu@43.201.233.185
```

Docker Swarm 초기화

Manager 노드에서 실행

```
# Swarm 초기화
docker swarm init --advertise-addr <MANAGER_NODE_IP>

# Join 토큰 확인
docker swarm join-token worker
```

Worker 노드에서 실행

```
# Manager 노드에서 출력된 join 명령어 실행
docker swarm join --token <TOKEN> <MANAGER_IP>:2377
```

Overlay 네트워크 생성

```
# traefik-public 네트워크 생성
docker network create --driver=overlay traefik-public

# staging-backend 네트워크 생성
docker network create --driver=overlay staging-backend
```

인증서 발급 및 Docker Secret 생성

```
# Traefik 스택 중지
docker stack rm traefik

# Certbot 설치
```

```
sudo apt-get update
sudo apt-get install certbot

# SSL 인증서 발급
sudo certbot certonly --standalone \
  -d on-gil.co.kr \
  -d www.on-gil.co.kr \
  -d staging.on-gil.co.kr \
  -d adminer.on-gil.co.kr \
  -d redis.on-gil.co.kr \
  -d rabbitmq.on-gil.co.kr \
  -d turn.on-gil.co.kr \
  --email your-email@example.com \
  --agree-tos \
  --no-eff-email

# Traefik 다시 시작
docker stack deploy -c traefik-stack.yml traefik

# SSL 인증서 secret 생성(모든 노드가 공유)
docker secret create traefik_cert /etc/letsencrypt/live/on-gil.co.kr/fullchain.pem
docker secret create traefik_key /etc/letsencrypt/live/on-gil.co.kr/privkey.pem
```

📌 4. Docker 컨테이너 생성

인프라 서버 (docker-compose.yml)

인프라 서버에서는 `docker-compose.yml`을 사용하여 모니터링 및 CI/CD 스택을 구성합니다.

위치: `/home/ubuntu/infra/docker-compose.yml`

주요 서비스:

- **Jenkins**: CI/CD 파이프라인
- **Prometheus**: 메트릭 수집
- **Grafana**: 모니터링 대시보드
- **Loki**: 로그 수집
- **Node Exporter**: 시스템 메트릭
- **cAdvisor**: 컨테이너 메트릭

실행 방법

```
cd /home/ubuntu/infra
docker-compose up -d
```

컨테이너 확인

```
docker-compose ps
```

서비스 서버 (staging-stack.yml)

서비스 서버(Docker Swarm)에서는 `staging-stack.yml`을 사용하여 애플리케이션 스택을 배포합니다.

위치: `/home/ubuntu/deploy/staging-stack.yml`

주요 서비스:

- **api**: Spring Boot 백엔드 애플리케이션 (3 replicas)
- **rabbitmq**: 메시지 브로커
- **adminer**: PostgreSQL 관리 도구
- **redisinsight**: Redis 관리 도구

스택 배포

```
cd /home/ubuntu/deploy
docker stack deploy -c staging-stack.yml ongil
```

스택 확인

```
# 스택 목록
docker stack ls

# 서비스 목록
docker stack services ongil

# 서비스 상태 확인
docker service ls

# 특정 서비스 로그 확인
docker service logs ongil_api -f
```

스택 업데이트

```
# 스택 제거
docker stack rm ongil

# 새로운 이미지 pull
docker pull ghcr.io/rabent/ongil-backend:latest

# 스택 재배포
docker stack deploy -c staging-stack.yml ongil
```

Traefik 스택 배포

위치: `/home/ubuntu/deploy/traefik-stack.yml`

```
docker stack deploy -c traefik-stack.yml traefik
```

Traefik 서비스 확인

```
docker service ls | grep traefik
```

📌 5. SSL 인증서 설정

인증서 자동 갱신

```
# Crontab 설정
sudo crontab -e

# 매월 1일 새벽 3시에 갱신 시도
0 3 1 * * certbot renew --quiet && docker secret rm traefik_cert traefik_key &&
docker secret create traefik_cert /etc/letsencrypt/live/on-gil.co.kr/fullchain.pem
&& docker secret create traefik_key /etc/letsencrypt/live/on-gil.co.kr/privkey.pem
&& docker service update --force traefik_traefik
```

📌 6. Jenkins 설정

Jenkins 접속

URL: `https://k13a305.p.ssafy.io/jenkin` (nginx proxy manager로 라우팅)

초기 비밀번호 확인

```
docker exec -it jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

Credentials 설정

Jenkins 관리 > Credentials에서 다음 항목 추가:

1. **GitLab API Token**
    - Kind: Secret text
    - ID: `gitlab-api-token`
2. **Backend .env 파일**

- Kind: Secret file
- ID: `backend-env`

3. **Docker Hub 로그인**

- Kind: Username with password
- ID: `dockerhub-credentials`
- Username: Docker Hub 사용자명
- Password: Docker Hub 비밀번호

4. **EC2 SSH Key**

- Kind: SSH Username with private key
- ID: `ec2-ssh-key`
- Username: `ubuntu`
- Private Key: PEM 파일 내용

Pipeline 생성

1. Backend CI Pipeline

- Pipeline name: `ongil-backend-ci`
- Build Triggers: GitLab webhook (MR to BE-dev)
- Pipeline script: 하단 스크립트 참조

2. Backend CD Pipeline

- Pipeline name: `ongil-backend-cd`
- Build Triggers: GitLab webhook (Push to BE-dev)
- Pipeline script: 하단 스크립트 참조

CI Pipeline Script (Backend)

```
<project>
<script/>
<script/>
<script/>
<actions/>
<description/>
<keepDependencies>false</keepDependencies>
<properties>
<com.dabsquared.gitlabjenkins.connection.GitLabConnectionProperty plugin="gitlab-
plugin@1.9.9">
<gitLabConnection>gitlab-connection</gitLabConnection>
<jobCredentialId/>
<useAlternativeCredential>false</useAlternativeCredential>
</com.dabsquared.gitlabjenkins.connection.GitLabConnectionProperty>
</properties>
<scm class="hudson.plugins.git.GitSCM" plugin="git@5.8.0">
<configVersion>2</configVersion>
<userRemoteConfigs>
```

```
<hudson.plugins.git.UserRemoteConfig>
<url>https://lab.ssafy.com/s13-final/S13P31A305.git</url>
<credentialsId>jenkins-personal</credentialsId>
</hudson.plugins.git.UserRemoteConfig>
</userRemoteConfigs>
<branches>
<hudson.plugins.git.BranchSpec>
<name>${source_branch}</name>
</hudson.plugins.git.BranchSpec>
</branches>
<doGenerateSubmoduleConfigurations>false</doGenerateSubmoduleConfigurations>
<submoduleCfg class="empty-list"/>
<extensions/>
</scm>
<canRoam>true</canRoam>
<disabled>false</disabled>
<blockBuildWhenDownstreamBuilding>false</blockBuildWhenDownstreamBuilding>
<blockBuildWhenUpstreamBuilding>false</blockBuildWhenUpstreamBuilding>
<triggers>
<org.jenkinsci.plugins.gwt.GenericTrigger plugin="generic-webhook-trigger@2.4.1">
<spec/>
<genericVariables>
<org.jenkinsci.plugins.gwt.GenericVariable>
<expressionType>JSONPath</expressionType>
<key>source_branch</key>
<value>$.object_attributes.source_branch</value>
<regexpFilter/>
<defaultValue/>
</org.jenkinsci.plugins.gwt.GenericVariable>
<org.jenkinsci.plugins.gwt.GenericVariable>
<expressionType>JSONPath</expressionType>
<key>target_branch</key>
<value>$.object_attributes.target_branch</value>
<regexpFilter/>
<defaultValue/>
</org.jenkinsci.plugins.gwt.GenericVariable>
<org.jenkinsci.plugins.gwt.GenericVariable>
<expressionType>JSONPath</expressionType>
<key>action</key>
<value>$.object_attributes.action</value>
<regexpFilter/>
<defaultValue/>
</org.jenkinsci.plugins.gwt.GenericVariable>
<org.jenkinsci.plugins.gwt.GenericVariable>
<expressionType>JSONPath</expressionType>
<key>mr_id</key>
<value>$.object_attributes.iid</value>
<regexpFilter/>
<defaultValue/>
</org.jenkinsci.plugins.gwt.GenericVariable>
</genericVariables>
<regexpFilterText>$target_branch $action</regexpFilterText>
<regexpFilterExpression>^backend (open|update|reopen)$</regexpFilterExpression>
<printPostContent>false</printPostContent>
```

```
<printContributedVariables>false</printContributedVariables>
<causeString>Generic Cause</causeString>
<token>back-ci</token>
<tokenCredentialId/>
<silentResponse>false</silentResponse>
<overrideQuietPeriod>false</overrideQuietPeriod>
<shouldNotFlattern>false</shouldNotFlattern>
<allowSeveralTriggersPerBuild>false</allowSeveralTriggersPerBuild>
</org.jenkinsci.plugins.gwt.GenericTrigger>
</triggers>
<concurrentBuild>false</concurrentBuild>
<builders>
<hudson.tasks.Shell>
<command>#!/bin/bash set -euo pipefail echo
"========================================" echo "Build #${BUILD_NUMBER}" echo
"========================================" cd backend echo "Step 1: Loading .env
from Jenkins credentials..." # Jenkins Secret File을 워크스페이스에 복사
(JENKINS_ENV_FILE은 임시 경로) ENV_FILE_PATH="$WORKSPACE/backend/.env" cp
"${ENV_FILE}" "${ENV_FILE_PATH}" chmod 600 "${ENV_FILE_PATH}" docker compose -f
docker-compose.CI.yml up -d --build docker cp backend-app-1:/app/build
./ongil/build echo "" echo "========================================" echo "✓
Build Completed Successfully!" echo "========================================"
</command>
<configuredLocalRules/>
</hudson.tasks.Shell>
<hudson.plugins.sonar.SonarRunnerBuilder plugin="sonar@2.18">
<project/>
<properties>sonar.projectBaseDir=/var/jenkins_home/workspace/CI/backend/ongil
sonar.projectKey=backend sonar.projectName=backend sonar.projectVersion=1.0
sonar.sources=src/main/java sonar.java.binaries=build/classes/java/main
sonar.sourceEncoding=UTF-8</properties>
<javaOpts/>
<additionalArguments/>
<jdk>jdk-17</jdk>
</hudson.plugins.sonar.SonarRunnerBuilder>
</builders>
<publishers>
<com.dabsquared.gitlabjenkins.publisher.GitLabCommitStatusPublisher
plugin="gitlab-plugin@1.9.9">
<name>jenkins</name>
<markUnstableAsSuccess>false</markUnstableAsSuccess>
</com.dabsquared.gitlabjenkins.publisher.GitLabCommitStatusPublisher>
</publishers>
<buildWrappers>
<hudson.plugins.ws__cleanup.PreBuildCleanup plugin="ws-cleanup@0.49">
<deleteDirs>false</deleteDirs>
<cleanupParameter/>
<externalDelete/>
<disableDeferredWipeout>false</disableDeferredWipeout>
</hudson.plugins.ws__cleanup.PreBuildCleanup>
<org.jenkinsci.plugins.credentialsbinding.impl.SecretBuildWrapper
plugin="credentials-binding@702.vfe613e537e88">
<bindings>
<org.jenkinsci.plugins.credentialsbinding.impl.FileBinding>
```

```
<credentialsId>.env</credentialsId>
<variable>ENV_FILE</variable>
</org.jenkinsci.plugins.credentialsbinding.impl.FileBinding>
</bindings>
</org.jenkinsci.plugins.credentialsbinding.impl.SecretBuildWrapper>
</buildWrappers>
</project>
```

CD Pipeline Script (Backend)

```
<project>
<script/>
<script/>
<script/>
<actions/>
<description/>
<keepDependencies>false</keepDependencies>
<properties>
<com.dabsquared.gitlabjenkins.connection.GitLabConnectionProperty plugin="gitlab-
plugin@1.9.9">
<gitLabConnection>gitlab-connection</gitLabConnection>
<jobCredentialId/>
<useAlternativeCredential>false</useAlternativeCredential>
</com.dabsquared.gitlabjenkins.connection.GitLabConnectionProperty>
</properties>
<scm class="hudson.plugins.git.GitSCM" plugin="git@5.8.0">
<configVersion>2</configVersion>
<userRemoteConfigs>
<hudson.plugins.git.UserRemoteConfig>
<url>https://lab.ssafy.com/s13-final/S13P31A305.git</url>
<credentialsId>jenkins-personal</credentialsId>
</hudson.plugins.git.UserRemoteConfig>
</userRemoteConfigs>
<branches>
<hudson.plugins.git.BranchSpec>
<name>backend</name>
</hudson.plugins.git.BranchSpec>
</branches>
<doGenerateSubmoduleConfigurations>false</doGenerateSubmoduleConfigurations>
<submoduleCfg class="empty-list"/>
<extensions/>
</scm>
<canRoam>true</canRoam>
<disabled>false</disabled>
<blockBuildWhenDownstreamBuilding>false</blockBuildWhenDownstreamBuilding>
<blockBuildWhenUpstreamBuilding>false</blockBuildWhenUpstreamBuilding>
<triggers>
<org.jenkinsci.plugins.gwt.GenericTrigger plugin="generic-webhook-trigger@2.4.1">
<spec/>
<genericVariables>
<org.jenkinsci.plugins.gwt.GenericVariable>
```

```
<expressionType>JSONPath</expressionType>
<key>source_branch</key>
<value>$.object_attributes.source_branch</value>
<regexpFilter/>
<defaultValue/>
</org.jenkinsci.plugins.gwt.GenericVariable>
<org.jenkinsci.plugins.gwt.GenericVariable>
<expressionType>JSONPath</expressionType>
<key>target_branch</key>
<value>$.object_attributes.target_branch</value>
<regexpFilter/>
<defaultValue/>
</org.jenkinsci.plugins.gwt.GenericVariable>
<org.jenkinsci.plugins.gwt.GenericVariable>
<expressionType>JSONPath</expressionType>
<key>action</key>
<value>$.object_attributes.action</value>
<regexpFilter/>
<defaultValue/>
</org.jenkinsci.plugins.gwt.GenericVariable>
<org.jenkinsci.plugins.gwt.GenericVariable>
<expressionType>JSONPath</expressionType>
<key>mr_id</key>
<value>$.object_attributes.iid</value>
<regexpFilter/>
<defaultValue/>
</org.jenkinsci.plugins.gwt.GenericVariable>
</genericVariables>
<regexpFilterText>$target_branch $action</regexpFilterText>
<regexpFilterExpression>^backend (merge|merged)$</regexpFilterExpression>
<printPostContent>false</printPostContent>
<printContributedVariables>false</printContributedVariables>
<causeString>Generic Cause</causeString>
<token>staging-cd</token>
<tokenCredentialId/>
<silentResponse>false</silentResponse>
<overrideQuietPeriod>false</overrideQuietPeriod>
<shouldNotFlattern>false</shouldNotFlattern>
<allowSeveralTriggersPerBuild>false</allowSeveralTriggersPerBuild>
</org.jenkinsci.plugins.gwt.GenericTrigger>
</triggers>
<concurrentBuild>false</concurrentBuild>
<builders>
<hudson.tasks.Shell>
<command>#!/bin/bash set -euo pipefail SERVER="ubuntu@13.125.112.26" echo
"========================================" echo "Build #${BUILD_NUMBER}" echo
"========================================" cd backend echo "Step 1: Loading .env
from Jenkins credentials..." # Jenkins Secret File을 워크스페이스에 복사
(JENKINS_ENV_FILE은 임시 경로) ENV_FILE_PATH="$WORKSPACE/backend/.env"
FIREBASE_PATH="$WORKSPACE/backend/ongil/src/main/resources/firebase" cp
"${ENV_FILE}" "${ENV_FILE_PATH}" cp "${FIREBASE}" "${FIREBASE_PATH}" chmod 600
"${ENV_FILE_PATH}" chmod 600 "${FIREBASE_PATH}" echo ${GITHUB_TOKEN} | docker
login ghcr.io -u rabent --password-stdin docker compose -f docker-
compose.stage.yml build && docker compose -f docker-compose.stage.yml push ssh -o
```

```
StrictHostKeyChecking=no ${SERVER} << ENDSSH echo ${GITHUB_TOKEN} | docker login
ghcr.io -u rabent --password-stdin docker stack deploy -c staging-stack.yml --
with-registry-auth staging sleep 10 docker stack ps staging ENDSSH echo "" echo
"========================================" echo "✓ Build Completed
Successfully!" echo "========================================"</command>
<configuredLocalRules/>
</hudson.tasks.Shell>
</builders>
<publishers/>
<buildWrappers>
<hudson.plugins.ws__cleanup.PreBuildCleanup plugin="ws-cleanup@0.49">
<deleteDirs>false</deleteDirs>
<cleanupParameter/>
<externalDelete/>
<disableDeferredWipeout>false</disableDeferredWipeout>
</hudson.plugins.ws__cleanup.PreBuildCleanup>
<org.jenkinsci.plugins.credentialsbinding.impl.SecretBuildWrapper
plugin="credentials-binding@702.vfe613e537e88">
<bindings>
<org.jenkinsci.plugins.credentialsbinding.impl.UsernamePasswordMultiBinding>
<credentialsId>docker_hub</credentialsId>
<usernameVariable>DOCKER_USER</usernameVariable>
<passwordVariable>DOCKER_PASS</passwordVariable>
</org.jenkinsci.plugins.credentialsbinding.impl.UsernamePasswordMultiBinding>
<org.jenkinsci.plugins.credentialsbinding.impl.FileBinding>
<credentialsId>.env</credentialsId>
<variable>ENV_FILE</variable>
</org.jenkinsci.plugins.credentialsbinding.impl.FileBinding>
<org.jenkinsci.plugins.credentialsbinding.impl.StringBinding>
<credentialsId>github_token</credentialsId>
<variable>GITHUB_TOKEN</variable>
</org.jenkinsci.plugins.credentialsbinding.impl.StringBinding>
<org.jenkinsci.plugins.credentialsbinding.impl.FileBinding>
<credentialsId>firebase_json</credentialsId>
<variable>FIREBASE</variable>
</org.jenkinsci.plugins.credentialsbinding.impl.FileBinding>
</bindings>
</org.jenkinsci.plugins.credentialsbinding.impl.SecretBuildWrapper>
<com.cloudbees.jenkins.plugins.sshagent.SSHAgentBuildWrapper plugin="ssh-
agent@386.v36cc0c7582f0">
<credentialIds>
<string>manager-pem</string>
</credentialIds>
<ignoreMissing>false</ignoreMissing>
</com.cloudbees.jenkins.plugins.sshagent.SSHAgentBuildWrapper>
</buildWrappers>
</project>
```

📌 7. 배포를 위한 파일 생성

Backend Dockerfile

위치: backend/ongil/Dockerfile

```
FROM gradle:8.14.3-jdk21 AS build

WORKDIR /app

# 그래들 파일 복사
COPY gradlew .
COPY gradle gradle
COPY build.gradle .
COPY settings.gradle .
# RUN apt-get update && apt-get install -y dos2unix && dos2unix gradlew
RUN chmod +x ./gradlew
RUN ./gradlew dependencies --no-daemon

COPY src src

# 권한 설정 및 빌드


RUN ./gradlew bootJar --no-daemon


FROM eclipse-temurin:21-jre-alpine

WORKDIR /app

# 빌드 단계에서 생성된 jar 파일 복사
COPY --from=build /app/build/libs/*.jar app.jar
COPY --from=build /app/build ./build

# 메모리 및 GC 최적화
ENV JAVA_OPTS="-Xms128m -Xmx256m -XX:+UseG1GC -XX:MaxGCPauseMillis=200"

# 애플리케이션 프로파일은 환경 변수로 지정
ENV SPRING_PROFILES_ACTIVE=default

ENV SERVER_PORT=8080

EXPOSE ${SERVER_PORT}

# 시작 명령어
ENTRYPOINT ["sh", "-c", "java $JAVA_OPTS -jar app.jar --
spring.profiles.active=$SPRING_PROFILES_ACTIVE --server.port=$SERVER_PORT"]
```

TURN Server Configuration

위치: /etc/turnserver.conf (서비스 서버 노드 중 택1)

```
# === Basic Listening Settings ===
listening-port=3478            # 기본 UDP/TCP 포트
tls-listening-port=5349        # TLS 포트
listening-ip=0.0.0.0

# === Relay Settings ===
relay-ip=172.31.45.62
external-ip=43.201.195.106/172.31.45.62
min-port=49152
max-port=65535
cert=/etc/letsencrypt/live/turn.on-gil.co.kr/fullchain.pem
pkey=/etc/letsencrypt/live/turn.on-gil.co.kr/privkey.pem
# === Authentication ===
use-auth-secret
static-auth-secret=A6BD47D478D827C8B49A4DED261E3
realm=turn.on-gil.co.kr

# === Security Settings ===
no-multicast-peers
no-cli
no-loopback-peers
fingerprint
stale-nonce=600
mobility

# === Logging ===
verbose
log-file=/var/log/turnserver/turnserver.log
log-binding

# === Performance Optimization ===
max-bps=0
bps-capacity=0
no-sslv2
no-sslv3
no-tlsv1
```

📌 8. EC2내 파일구조

인프라 서버

```
/home/ubuntu/
├── infra/
│   ├── docker-compose.yml          # 인프라 스택 정의
│   ├── prometheus/
│   │   └── prometheus.yml          # Prometheus 설정
│   ├── loki/
│   │   └── loki-config.yml        # Loki 설정
│   └── grafana/
```

```
│       └── provisioning/          # Grafana 대시보드 설정
└── jenkins_data/                  # Jenkins 영구 저장소
```

서비스 서버 (Swarm Manager)

```
/home/ubuntu/
├── deploy/
│   ├── traefik-stack.yml          # Traefik 리버스 프록시
│   ├── traefik-dynamic.yml        # Traefik 동적 설정
│   ├── staging-stack.yml          # 애플리케이션 스택
│   └── .env                       # 환경 변수 (배포 시 복사)
├── /etc/letsencrypt/              # SSL 인증서
│   └── live/
│       └── on-gil.co.kr/
│           ├── fullchain.pem
│           └── privkey.pem
└── logs/                          # 애플리케이션 로그
```

각 설정 파일 내용

prometheus.yml:

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s
  external_labels:
    cluster: 'local'
    environment: 'dev'

# Alertmanager configuration (optional)
# alerting:
#   alertmanagers:
#     - static_configs:
#         - targets:
#           - alertmanager:9093

# 알림 규칙 파일 (optional)
# rule_files:
#   - "rules/*.yml"

scrape_configs:
  # Prometheus 자체 메트릭
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']

  # Node Exporter (서버 메트릭)
  - job_name: 'node-exporter'
    static_configs:
```

```
      - targets: ['node-exporter:9100']
        labels:
          env: 'infra'
          server_type: 'infrastructure'
          server_name: 'infra-server'

  # cAdvisor (Docker 컨테이너 메트릭)
  - job_name: 'cadvisor'
    static_configs:
      - targets: ['cadvisor:8080']
        labels:
          env: 'infra'
          server_type: 'infrastructure'
          server_name: 'infra-server'

  # Grafana
  - job_name: 'grafana'
    static_configs:
      - targets: ['grafana:3000']

  # Loki
  - job_name: 'loki'
    static_configs:
      - targets: ['loki:3100']

  - job_name: 'swarm-cadvisor'
    static_configs:
      - targets:
        - '13.125.112.26:8080'
        labels:
          env: 'production'
          server_type: 'swarm'
          server_name: 'swarm-node-1'
      - targets:
        - '43.201.195.106:8080'
        labels:
          env: 'production'
          server_type: 'swarm'
          server_name: 'swarm-node-2'
      - targets:
        - '43.201.233.185:8080'
        labels:
          env: 'production'
          server_type: 'swarm'
          server_name: 'swarm-node-3'

  - job_name: 'swarm-node'
    static_configs:
      - targets:
        - '13.125.112.26:9100'
        labels:
          env: 'production'
          server_type: 'swarm'
          server_name: 'swarm-node-1'
```

```
      - targets:
        - '43.201.195.106:9100'
        labels:
          env: 'production'
          server_type: 'swarm'
          server_name: 'swarm-node-2'
      - targets:
        - '43.201.233.185:9100'
        labels:
          env: 'production'
          server_type: 'swarm'
          server_name: 'swarm-node-3'

  - job_name: 'swarm-traefik'
    static_configs:
      - targets:
        - '13.125.112.26:8082'
        labels:
          env: 'production'
          server_type: 'swarm'
          server_name: 'swarm-node-1'
      - targets:
        - '43.201.195.106:8082'
        labels:
          env: 'production'
          server_type: 'swarm'
          server_name: 'swarm-node-2'
      - targets:
        - '43.201.233.185:8082'
        labels:
          env: 'production'
          server_type: 'swarm'
          server_name: 'swarm-node-3'
```

loki-config.yml:

```
auth_enabled: false

server:
  http_listen_port: 3100
  grpc_listen_port: 9096
  http_listen_address: 0.0.0.0
  grpc_listen_address: 0.0.0.0

common:
  instance_addr: 127.0.0.1
  path_prefix: /loki
  storage:
    filesystem:
      chunks_directory: /loki/chunks
      rules_directory: /loki/rules
  replication_factor: 1
```

```
  ring:
    kvstore:
      store: inmemory

query_range:
  results_cache:
    cache:
      embedded_cache:
        enabled: true
        max_size_mb: 100

schema_config:
  configs:
    - from: 2024-01-01
      store: tsdb
      object_store: filesystem
      schema: v13
      index:
        prefix: index_
        period: 24h

limits_config:
  retention_period: 744h
  allow_structured_metadata: true
  max_query_series: 500

compactor:
  working_directory: /loki/compactor
  compaction_interval: 10m
  retention_enabled: true
  retention_delete_delay: 2h
  retention_delete_worker_count: 150
  delete_request_store: filesystem
```

traefik-stack.yml:

```
version: '3.8'

services:
  traefik:
    image: traefik:v2.10
    command:
      # Docker Swarm 설정
      - --providers.docker=true
      - --providers.docker.swarmMode=true
      - --providers.docker.exposedByDefault=false
      - --providers.docker.network=traefik-public
      - --providers.docker.watch=true

      # 엔트리포인트
      - --entrypoints.web.address=:80
      - --entrypoints.websecure.address=:443
```

```
      - --entrypoints.metrics.address=:8082
      - --metrics.prometheus=true
      - --metrics.prometheus.entryPoint=metrics
      - --metrics.prometheus.addEntryPointsLabels=true
      - --metrics.prometheus.addRoutersLabels=true
      - --metrics.prometheus.addServicesLabels=true

      # Let's Encrypt - HTTP Challenge
      #- --certificatesresolvers.letsencrypt.acme.email=rabent0207@gmail.com
      #- --certificatesresolvers.letsencrypt.acme.storage=/letsencrypt/acme.json
      #- --certificatesresolvers.letsencrypt.acme.httpchallenge=true
      #- --certificatesresolvers.letsencrypt.acme.httpchallenge.entrypoint=web

      # ⭐ Secret 사용으로 변경
      - --providers.file.directory=/etc/traefik/dynamic
      - --providers.file.watch=true

      # 로그
      - --log.level=INFO
      - --accesslog=true

      # Ping 엔드포인트 (헬스체크용)
      - --ping=true

      # API/Dashboard (선택사항)
      - --api.dashboard=true

    ports:
      - target: 80
        published: 80
        mode: host
      - target: 443
        published: 443
        mode: host
      - target: 8082
        published: 8082
        mode: host

    volumes:
      - /var/run/docker.sock:/var/run/docker.sock:ro
      # - traefik-certificates:/letsencrypt

    secrets:
      - traefik_cert
      - traefik_key

    configs:
      - source: traefik_dynamic
        target: /etc/traefik/dynamic/certs.yml

    networks:
      - traefik-public

    deploy:
```

```
      # replicas: 1  # ☑ 1개만 실행
      mode: global
      placement:
        constraints:
          - node.role == manager  # manager 노드에만 배치
        preferences:
          - spread: node.id  # 여러 manager 중 분산 배치
      update_config:
        parallelism: 1
        delay: 10s
        order: start-first
      restart_policy:
        condition: on-failure
        delay: 5s
        max_attempts: 3
      labels:
        # Dummy service (Traefik 자체는 포트 노출 안 함)
        - traefik.http.services.dummy.loadbalancer.server.port=8080

    healthcheck:
      test: ["CMD", "traefik", "healthcheck", "--ping"]
      interval: 10s
      timeout: 3s
      retries: 3

  cadvisor:
    image: gcr.io/cadvisor/cadvisor:latest
    volumes:
      - /:/rootfs:ro
      - /var/run:/var/run:ro
      - /sys:/sys:ro
      - /var/lib/docker/:/var/lib/docker:ro
    ports:
      - "8080:8080"
    deploy:
      mode: global  # 모든 노드에 하나씩
      resources:
        limits:
          memory: 128M
    networks:
      - traefik-public

  node-exporter:
    image: prom/node-exporter:latest
    volumes:
      - /proc:/host/proc:ro
      - /sys:/host/sys:ro
      - /:/rootfs:ro
    command:
      - '--path.procfs=/host/proc'
      - '--path.sysfs=/host/sys'
      - '--collector.filesystem.mount-points-exclude=^/(sys|proc|dev|host|etc)
($$|/)'
    ports:
```

```
      - "9100:9100"
    deploy:
      mode: global
    networks:
      - traefik-public


volumes:
  traefik-certificates:

secrets:
  traefik_cert:
    external: true
  traefik_key:
    external: true

configs:
  traefik_dynamic:
    file: ./traefik-dynamic.yml

networks:
  traefik-public:
    external: true
```

traefik-dynamic.yml:

```
tls:
  certificates:
    - certFile: /run/secrets/traefik_cert
      keyFile: /run/secrets/traefik_key
      stores:
        - default

  stores:
    default:
      defaultCertificate:
        certFile: /run/secrets/traefik_cert
        keyFile: /run/secrets/traefik_key
```

staging-stack.yml:

```
version: '3.8'

services:
  adminer:
    image: adminer:latest
    environment:
      ADMINER_DEFAULT_SERVER: a305-db.cz22acwewmnw.ap-northeast-
```

```
2.rds.amazonaws.com
      ADMINER_DESIGN: nette  # 테마 변경
    networks:
      - staging-backend
      - traefik-public
    deploy:
      replicas: 1
      placement:
        constraints:
          - node.hostname == ip-172-31-47-15
      labels:
        - "traefik.enable=true"
        - "traefik.docker.network=traefik-public"

        # ☑ 서브도메인 사용 (권장)
        - "traefik.http.routers.adminer.rule=Host(`adminer.on-gil.co.kr`)"
        - "traefik.http.routers.adminer.entrypoints=websecure"
        - "traefik.http.routers.adminer.tls=true"
        # - "traefik.http.routers.adminer.tls.certresolver=letsencrypt"
        - "traefik.http.services.adminer.loadbalancer.server.port=8080"

  redisinsight:
    image: redis/redisinsight:latest
    networks:
      - staging-backend
      - traefik-public
    deploy:
      replicas: 1
      placement:
        constraints:
          - node.hostname == ip-172-31-47-15
      labels:
        - "traefik.enable=true"
        - "traefik.docker.network=traefik-public"

        # HTTP 라우터 (디버깅용)
        - "traefik.http.routers.redisinsight-http.rule=Host(`redis.on-gil.co.kr`)"
        - "traefik.http.routers.redisinsight-http.entrypoints=web"
        - "traefik.http.routers.redisinsight-http.service=redisinsight"

        # HTTPS 라우터
        - "traefik.http.routers.redisinsight.rule=Host(`redis.on-gil.co.kr`)"
        - "traefik.http.routers.redisinsight.entrypoints=websecure"
        - "traefik.http.routers.redisinsight.tls=true"
        # - "traefik.http.routers.redisinsight.tls.certresolver=letsencrypt"
        - "traefik.http.routers.redisinsight.service=redisinsight"

        # 🔥 핵심: 서비스 포트 설정
        - "traefik.http.services.redisinsight.loadbalancer.server.port=5540"


  rabbitmq:
    image: my-rabbitmq:stomp
    environment:
```

```
      - RABBITMQ_DEFAULT_USER=admin
      - RABBITMQ_DEFAULT_PASS=password
    networks:
      - staging-backend
      - traefik-public
    volumes:
      - rabbitmq-data:/var/lib/rabbitmq
    deploy:
      replicas: 1
      placement:
        constraints:
          - node.hostname == ip-172-31-47-15
      labels:
        - "traefik.enable=true"
        - "traefik.docker.network=traefik-public"
        - "traefik.http.routers.rabbitmq-http.rule=Host(`rabbitmq.on-gil.co.kr`)"
        - "traefik.http.routers.rabbitmq-http.entrypoints=web"
        - "traefik.http.routers.rabbitmq-http.service=rabbitmq"
        - "traefik.http.routers.rabbitmq.rule=Host(`rabbitmq.on-gil.co.kr`)"
        - "traefik.http.routers.rabbitmq.entrypoints=websecure"
        - "traefik.http.routers.rabbitmq.tls=true"
        # - "traefik.http.routers.rabbitmq.tls.certresolver=letsencrypt"
        - "traefik.http.services.rabbitmq.loadbalancer.server.port=15672"

  api:
    image: ghcr.io/rabent/ongil-backend:latest   # staging 태그 사용 권장
    networks:
      - traefik-public
      - staging-backend
    healthcheck:
      test: ["CMD", "wget", "--no-verbose", "--tries=1", "--spider",
"http://localhost:8080/actuator/health"]
      interval: 10s
      timeout: 3s
      retries: 3
      start_period: 40s
    environment:
      TZ: Asia/Seoul
      JAVA_TOOL_OPTIONS: -Duser.timezone=Asia/Seoul
      # Database
      DB_HOST: a305-db.cz22acwewmnw.ap-northeast-2.rds.amazonaws.com  # 서비스 이름
으로 연결
      DB_PORT: 5432
      DB_NAME: ongil
      DB_USERNAME: postgres
      DB_PASSWORD: password
      DB_SSL_MODE: require  # 내부 네트워크이므로 SSL 불필요
      DB_POOL_MAX_SIZE: 10
      DB_POOL_MIN_IDLE: 2

      # CORS - 개발용: 모든 origin 허용
      CORS_ALLOWED_ORIGINS: https://on-gil.co.kr,https://www.on-
gil.co.kr,https://staging.on-gil.co.kr
      CORS_ALLOWED_METHODS: GET,POST,PUT,DELETE,PATCH,OPTIONS
```

```
      CORS_ALLOWED_HEADERS: Authorization,Content-Type,X-Requested-With
      CORS_ALLOW_CREDENTIALS: true
      CORS_MAX_AGE: 3600

      # ========= GMS =========
      GMS_API_KEY: key


      # Redis
      REDIS_HOST: a305-cache-ui6bqx.serverless.apn2.cache.amazonaws.com  # 서비스
이름으로 연결
      REDIS_PORT: 6379
      REDIS_PASSWORD:
      REDIS_SSL_ENABLED: "true"  # 내부 네트워크이므로 SSL 불필요

      AWS_REGION: ap-northeast-2
      AWS_ACCESS_KEY: key
      AWS_SECRET_KEY: key
      S3_BUCKET: a305-bucket
      JWT_SECRET: key
      JWT_ACCESS_TOKEN_EXPIRATION: 86400000        # 24h
      JWT_REFRESH_TOKEN_EXPIRATION: 259200000      # 72h


      # ========= TMAP API =========
      TMAP_APP_KEY: key


      # ========= COOL SMS =========
      COOLSMS_API_KEY: key
      COOLSMS_API_SECRET: key
      COOLSMS_FROM_NUMBER: "01053451951"  # 발신번호로 승인된 번호

      # ========= Turn Stun =========
      TURN_SHARED_SECRET: key
      TURN_SERVER_HOST: turn.on-gil.co.kr
      TURN_SERVER_PORT: 3478
      TURN_TTL: 3600

      # ========= Call API =========
      CALL_TIMEOUT_SECONDS: 60
      SPRING_PROFILES_ACTIVE: dev
      # Server
      SERVER_PORT: 8080

      RABBITMQ_HOST: rabbitmq
      RABBITMQ_PORT: 5672
      RABBITMQ_STOMP_PORT: 61613
      RABBITMQ_USERNAME: admin
      RABBITMQ_PASSWORD: password

    deploy:
      replicas: 3
      labels:
```

```
        - traefik.enable=true
        - traefik.docker.network=traefik-public
        - traefik.http.routers.staging-api.rule=Host(`staging.on-gil.co.kr`) &&
(PathPrefix(`/api`) || PathPrefix(`/swagger-ui`) || PathPrefix(`/v3/api-docs`) ||
PathPrefix(`/ws`))
        - traefik.http.routers.staging-api.entrypoints=websecure
        - traefik.http.routers.staging-api.tls=true
        # - traefik.http.routers.staging-api.tls.certresolver=letsencrypt
        - traefik.http.services.staging-api.loadbalancer.server.port=8080

        - traefik.http.routers.staging-api-http.rule=Host(`staging.on-gil.co.kr`)
&& (PathPrefix(`/api`) || PathPrefix(`/swagger-ui`) || PathPrefix(`/v3/api-docs`)
|| PathPrefix(`/ws`))
        - traefik.http.routers.staging-api-http.entrypoints=web
        - traefik.http.routers.staging-api-http.service=staging-api
        #- traefik.http.middlewares.staging-api-strip.stripprefix.prefixes=/api
        #- traefik.http.routers.staging-api.middlewares=staging-api-strip
      placement:
        max_replicas_per_node: 1
      restart_policy:
        condition: on-failure
        delay: 5s
        max_attempts: 3
    logging:
      driver: json-file
      options:
        max-size: "10m"    # 파일당 최대 10MB
        max-file: "3"      # 최대 3개 파일 보관
        labels: "app.name,app.version"  # 라벨도 로그에 포함

networks:
  traefik-public:
    external: true
  staging-backend:
    driver: overlay

volumes:
  redisinsight-data:
  rabbitmq-data:
```

docker-compose.yml :

```
version: '3.8'

services:
  # ============================================# CI/CD#
==========================================

  jenkins:
    image: jenkins/jenkins:lts
    container_name: jenkins
    restart: unless-stopped
```

```
    user: root
    ports:
      - "8080:8080"      # Jenkins UI
      - "50000:50000"    # Jenkins Agent
    volumes:
      - ./jenkins_home:/var/jenkins_home
      - /var/run/docker.sock:/var/run/docker.sock
      - /usr/bin/docker:/usr/bin/docker                # ← 추가
      - /usr/libexec/docker:/usr/libexec/docker        # (시스템에 따라 필요)
    environment:
    - JENKINS_OPTS=--prefix=/jenkins
    networks:
      - monitoring

  sonarqube:
    image: sonarqube:community
    container_name: sonarqube
    restart: unless-stopped
    ports:
      - "9000:9000"
    environment:
      - SONAR_JDBC_URL=jdbc:postgresql://sonarqube-db:5432/sonar
      - SONAR_JDBC_USERNAME=sonar
      - SONAR_JDBC_PASSWORD=sonar
      - SONAR_WEB_CONTEXT=/sonarqube
    volumes:
      - sonarqube_data:/opt/sonarqube/data
      - sonarqube_extensions:/opt/sonarqube/extensions
      - sonarqube_logs:/opt/sonarqube/logs
    depends_on:
      - sonarqube-db
    networks:
      - monitoring

  sonarqube-db:
    image: postgres:15-alpine
    container_name: sonarqube-db
    restart: unless-stopped
    environment:
      - POSTGRES_USER=sonar
      - POSTGRES_PASSWORD=sonar
      - POSTGRES_DB=sonar
    volumes:
      - sonarqube_db:/var/lib/postgresql/data
    networks:
      - monitoring

  nginx-proxy-manager:
    image: jc21/nginx-proxy-manager:latest
    container_name: nginx-proxy-manager
    restart: unless-stopped
    ports:
      - "80:80"      # HTTP
      - "443:443"    # HTTPS
```

```
      - "8081:81"      # Admin UI
    environment:
      DB_SQLITE_FILE: "/data/database.sqlite"
    volumes:
      - npm_data:/data
      - npm_letsencrypt:/etc/letsencrypt
    networks:
      - monitoring

  # ============================================# Monitoring - Metrics#
==========================================

  prometheus:
    image: prom/prometheus:latest
    container_name: prometheus
    restart: unless-stopped
    ports:
      - "9090:9090"
    command:
      - '--config.file=/etc/prometheus/prometheus.yml'
      - '--storage.tsdb.path=/prometheus'
      - '--storage.tsdb.retention.time=15d'
      - '--web.enable-lifecycle'
    volumes:
      - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml
      - prometheus_data:/prometheus
    networks:
      - monitoring

  grafana:
    image: grafana/grafana:latest
    container_name: grafana
    restart: unless-stopped
    ports:
      - "3000:3000"
    environment:
      - GF_SECURITY_ADMIN_USER=admin
      - GF_SECURITY_ADMIN_PASSWORD=admin
      - GF_INSTALL_PLUGINS=grafana-piechart-panel
      - GF_SERVER_ROOT_URL=https://k13a305.p.ssafy.io/grafana
      - GF_SERVER_SERVE_FROM_SUB_PATH=true
    volumes:
      - grafana_data:/var/lib/grafana
      - ./grafana/provisioning:/etc/grafana/provisioning
    depends_on:
      - prometheus
    networks:
      - monitoring

  # ============================================# Monitoring - Logs#
==========================================

  loki:
    image: grafana/loki:latest
```

```
    container_name: loki
    restart: unless-stopped
    ports:
      - "3100:3100"
    command: -config.file=/etc/loki/local-config.yaml
    volumes:
      - ./loki/loki-config.yml:/etc/loki/local-config.yaml
      - loki_data:/loki
    networks:
      - monitoring

  # ==========================================# Exporters#
==========================================

  node-exporter:
    image: prom/node-exporter:latest
    container_name: node-exporter
    restart: unless-stopped
    ports:
      - "9100:9100"
    command:
      - '--path.procfs=/host/proc'
      - '--path.sysfs=/host/sys'
      - '--path.rootfs=/rootfs'
      - '--collector.filesystem.mount-points-exclude=^/(sys|proc|dev|host|etc)
($$|/)'
    volumes:
      - /proc:/host/proc:ro
      - /sys:/host/sys:ro
      - /:/rootfs:ro
    networks:
      - monitoring

  cadvisor:
    image: gcr.io/cadvisor/cadvisor:latest
    container_name: cadvisor
    restart: unless-stopped
    user: root
    ports:
      - "8082:8080"
    volumes:
      - /:/rootfs:ro
      - /var/run:/var/run:ro
      - /sys:/sys:ro
      - /var/lib/docker/:/var/lib/docker:ro
      - /dev/disk/:/dev/disk:ro
      - /var/run/docker.sock:/var/run/docker.sock:ro
    privileged: true
    networks:
      - monitoring


networks:
  monitoring:
```

```
    driver: bridge

volumes:
  # CI/CD
  sonarqube_data:
  sonarqube_extensions:
  sonarqube_logs:
  sonarqube_db:
  npm_data:
  npm_letsencrypt:

  # Monitoring
  prometheus_data:
  grafana_data:
  loki_data:
```

모니터링 접속 정보

- **Grafana**: `https://k13a305.p.ssafy.io/grafana` (admin/admin)
- **Prometheus**: `http://k13a305.p.ssafy.io:prometheus`
- **Jenkins**: `https://k13a305.p.ssafy.io/jenkins`
- **Adminer**: `https://adminer.on-gil.co.kr`
- **RedisInsight**: `https://redis.on-gil.co.kr`
- **RabbitMQ**: `https://rabbitmq.on-gil.co.kr` (admin/password)
